

APEX Plugin Architecture (Code-Accurate)

Scope: backend/src/plugins/, backend/src/interfaces/, PluginRegistry, SessionService freeze logic, ScoringService.generate_feedback.

Why Plugins Exist

- **Patient text** is produced by classes implementing **PatientModel** — not by hard-coded calls inside **DialogueService**.
- **Post-session feedback** is produced by classes implementing **Evaluator**, each delegating to a specific **ScoringService** method (**rule_only**, **hybrid**, **hybrid_v2**).
- **Extra metrics** after evaluation implement **MetricsPlugin.compute** and fill **sessions.metrics_json**.

Protocols (interfaces/)

Protocol	Method	Return
interfaces/patient_model.py PatientModel	async def generate_response(self, state: Any, clinician_input: str) -> str	Patient utterance
interfaces/evaluator.py Evaluator	async def evaluate(self, db, session_id: int) -> FeedbackResponse	API feedback DTO
interfaces/metrics.py MetricsPlugin	def compute(self, db, session_id: int) -> dict[str, Any]	Arbitrary JSON-serializable dict

Startup Loading

core/events.py → load_plugins() in plugins/load_plugins.py:

```
PLUGIN_MODULES = [  
    "plugins.patient_models.default_llm_patient",  
    "plugins.evaluators.apex_baseline_evaluator",  
    "plugins.evaluators.apex_hybrid_evaluator",  
    "plugins.evaluators.apex_hybrid_v2_evaluator",  
    "plugins.metrics.apex_metrics",  
]
```

Each module **imports** and calls **PluginRegistry.register_*** with a class-level **name** string (the registry key).
There is **no** filesystem scan: new plugins **must** be importable modules in this list (or imported as a side effect of a listed module).

Registry (plugins/registry.py)

- register_evaluator / get_evaluator / list_evaluators
- register_patient_model / get_patient_model / list_patient_models
- register_metrics_plugin / get_metrics_plugin / list_metrics_plugins

(Note: method name is **register_metrics_plugin**, singular.)

When Plugins Run

Type	When	Resolver
PatientModel	Every DialogueService.process_user_turn	session.patient_model_plugin → else settings.patient_model_plugin; get or _load_class_from_path + register_patient_model
Evaluator	ScoringService.generate_feedback only	session.evaluator_plugin → else settings.evaluator_plugin; get or load + register_evaluator
MetricsPlugin	Immediately after evaluator.evaluate in generate_feedback	_run_and_store_metrics_plugins: parse session.metrics_plugins JSON list; get_metrics_plugin or load + register_metrics_plugin

If **metrics_plugins** is empty / null, **metrics_json** is not updated by this path.
Internal helpers **generate_feedback_rule_only**, **generate_feedback_hybrid**, **generate_feedback_hybrid_v2** do **not** call **_run_and_store_metrics_plugins**; metrics run only when scoring goes through **generate_feedback** (the normal close-session path).

Frozen Configuration (SessionService.create_session)

Resolution order **evaluator**:

1. `SessionCreate.evaluator_plugin` (request body), if non-empty string
2. else `case.evaluator_plugin`
3. else `settings.evaluator_plugin`

If request or case specifies an unknown evaluator → **HTTP 400** "Invalid evaluator plugin".
If only settings default and class not in registry → `_load_class_from_path` + register (backward compatibility).

Patient model: `case.patient_model_plugin` else `settings.patient_model_plugin`. Invalid **case** override → **400**; settings-only unknown path → dynamic load + register.

Metrics: `case.metrics_plugins` (JSON text → list) else `settings.metrics_plugins`. Invalid **case** list → **400**; settings-only unknown → dynamic load + register.

Stored on **Session**: `evaluator_plugin`, `evaluator_version`, `patient_model_plugin`, `patient_model_version`, `metrics_plugins` (JSON text), later `metrics_json`.

Bundled Plugin Classes

Class	Module	Registry name (equals settings default where applicable)
<code>DefaultLLMPatientModel</code>	<code>plugins/patient_models/default_llm_patient.py</code>	<code>plugins.patient_models.default_llm_patient:DefaultLLMPatientMod</code>
<code>ApexBaselineEvaluator</code>	<code>plugins/evaluators/apex_baseline_evaluator.py</code>	<code>plugins.evaluators.apex_baseline_evaluator:ApexBaselineEvaluato</code>
<code>ApexHybridEvaluator</code>	<code>plugins/evaluators/apex_hybrid_evaluator.py</code>	<code>plugins.evaluators.apex_hybrid_evaluator:ApexHybridEvaluator</code>
<code>ApexHybridV2Evaluator</code>	<code>plugins/evaluators/apex_hybrid_v2_evaluator.py</code>	<code>plugins.evaluators.apex_hybrid_v2_evaluator:ApexHybridV2Evaluat</code>
<code>ApexMetrics</code>	<code>plugins/metrics/apex_metrics.py</code>	<code>plugins.metrics.apex_metrics:ApexMetrics</code>

Each class defines **name** and **version** attributes.

DefaultLLMPatientModel uses `get_settings().default_llm_provider: "gemini"` → `GeminiAdapter()`, otherwise `OpenAIAdapter()`, then `PatientPromptBuilder.build_prompt` and `LLMAdapter.generate_patient_response`.

ApexMetrics.compute calls **ScoringService** helpers such as `_extract_spans_from_turns`, `_calculate_eo_counts_by_dimension`, `_calculate_response_counts_by_type`, `_analyze_spikes_coverage` (relies on **private** methods — acceptable for in-repo plugins, fragile for external forks).

Admin Discovery Endpoints

- `GET /v1/admin/plugin-registry` — lists registered plugins with **name** + **version** from **PluginRegistry**.
- `GET /v1/admin/plugins` — returns **settings** defaults only (`patient_model_plugin`, `evaluator_plugin`, `metrics_plugins`), not per-session freeze.

Example Execution

1. Session created with default settings → `evaluator ...:ApexHybridEvaluator`, `metrics [...:ApexMetrics]`.
2. Trainee chats; each turn uses `DefaultLLMPatientModel.generate_response`.
3. `POST ...:close` → `ApexHybridEvaluator.evaluate` → `ScoringService.generate_feedback_hybrid` (persists feedback with rule+LLM merge as implemented).
4. `generate_feedback` then runs `ApexMetrics.compute` → merges into `sessions.metrics_json` under key `plugins.metrics.apex_metrics:ApexMetrics`.

Design Constraints (From Implementation)

- Plugin ids are strings `module.path:ClassName` validated in `config/settings.py` for default fields.
- **FeedbackResponse** / turn schemas are **not** plugin-extensible without code changes.
- Evaluators are expected to persist feedback through **ScoringService** helpers; custom evaluators should follow the same persistence pattern or feedback rows may be missing.

Developer steps — `03_Developer_Quickstart.md`